Software Development

Life Cycle (SDLC)

From Beginner to Expert

📒 What is SDLC? | 7 Phases | 6 Key Models | Real-World Use Cases

Session Roadmap — 1 Hour

0–10 min	10–25 min	25–40 min	40–55 min	55–60 min
1	2	3	4	5
SDLC Basics	**Classic Models**	**Iterative Models**	**Agile & DevOps**	**Wrap-Up & Q&A**
Definition, why it matters, the 7 phases	Waterfall, V-Model — linear approaches	Incremental, Spiral, RAD — adaptive approaches	Scrum, Kanban, CI/CD — modern practices	Model comparison table + key takeaways

What is SDLC?

Definition

SDLC is a structured framework used by software organizations to design, develop, and test high-quality software.

It defines the entire process of software production — from the initial idea to final deployment and maintenance.

Goal: Deliver software that meets expectations, stays on budget, and is easy to maintain.

👁‍🗨 Visibility

Stakeholders track every stage

🔍 Quality Control

Testing is built-in, not bolted-on

🛡 ⚠ Risk Management

Identify pitfalls in the planning phase

💰 Cost Estimation

Accurate timelines & budget prediction

"Without SDLC, software development becomes chaotic."

The 7 Phases of SDLC

1

Planning

Feasibility, resources, schedule, cost estimation

2

Requirements

SRS document — functional & non-functional specs

3

Design

HLD + LLD — architecture, DB schema, APIs

4

Development

Coding with IDEs, Git, code reviews, unit tests

5

Testing

Unit → Integration → System → UAT

6

Deployment

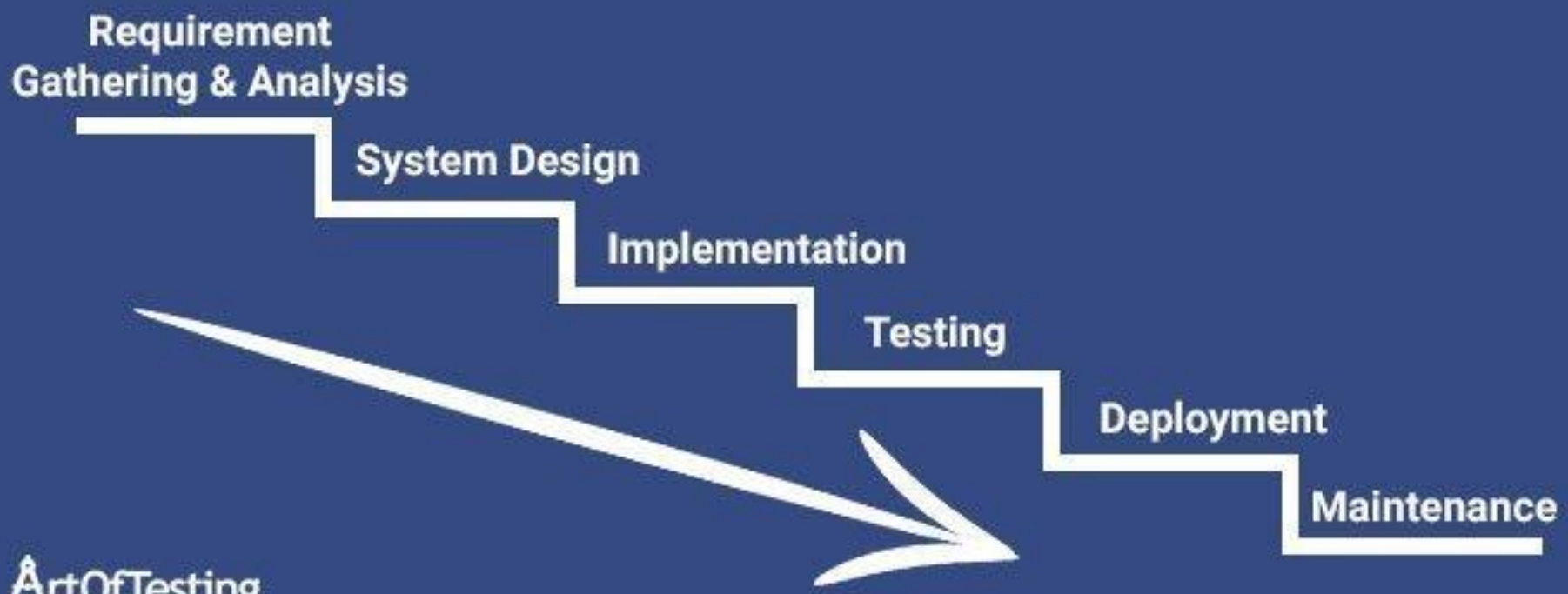
CI/CD pipelines, prod setup, smoke testing

7

Maintenance

Bug fixes, patches, upgrades, new features

Waterfall Model



ArtOfTesting

Definition

The **Waterfall Model** is a **linear and sequential software development model** where each phase must be completed before moving to the next phase. The output of one phase becomes the input for the next phase.

Main Characteristics

- **Sequential process** – Development flows step by step like a waterfall.
- **No overlapping phases** – Each stage must finish before the next begins.
- **Strong documentation** – Detailed documents are created in every phase.
- **Clear milestones** – Easy to track project progress.
- **Minimal changes allowed** once a phase is completed.

Disadvantages:

- Very **rigid and inflexible**.
- Hard to go back to previous phases.
- Changes are difficult once development starts.
- Not suitable for large or complex projects

When to Use Waterfall Model

- Requirements are **clear and stable**.
- Project is **small or medium-sized**.
- Risks are **low and predictable**.
- Detailed documentation is required (e.g., government or regulated systems).

Model 1: Waterfall

CLASSIC

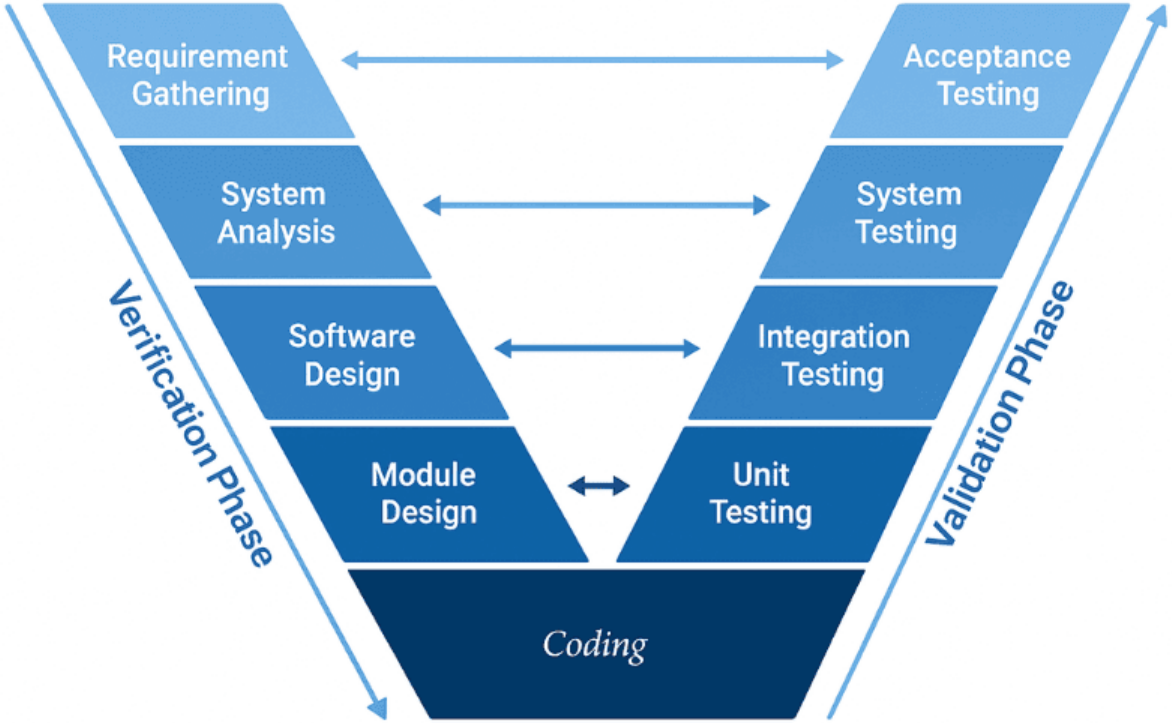
1. Requirements
2. System Design
3. Implementation
4. Testing
5. Deployment
6. Maintenance

"Sequential, disciplined, document-heavy — perfect for stable projects"

Key Facts

- Simple, easy to understand
- Clear milestones & documentation
- No flexibility — changes are costly
- Customer sees product only at the end
- Best for: fixed requirements (gov, defense)
- Speed: Slow | Feedback: At the end

V-MODEL


Requirement Gathering
Acceptance Testing
System Analysis
System Testing
Software Design
Integration Testing
Module Design
Unit Testing
Coding
Verification Phase
Validation Phase

The **V-Model (Verification and Validation Model)** is an extension of the **Waterfall Model** where **testing is planned alongside each development phase**.
It is called the **V-Model** because the development stages go down the left side of the “V”, and testing stages go up the right side.

Key Concept:

- **Verification** → Checking if the product is built correctly according to design.
- **Validation** → Checking if the correct product is built for the user.
- Each development stage has a **corresponding testing phase**.

Key Characteristics

- Testing is **planned early** in development.
- Every development stage has a **matching testing stage**.
- Strong **documentation and quality control**.

Helps detect defects **early in the development process**.

Advantages

Early detection of errors.
Clear structure and easy to manage.
Testing activities are well planned.
Suitable for **small and medium projects with clear requirements**.

Disadvantages

- Very **rigid and not flexible**.
- Difficult to handle changing requirements.
- Not suitable for **large or complex projects**.
- No working software until late in the process.

When to Use V-Model

- Requirements are **clearly defined**.
- Project has **high reliability requirements**.
- Used in systems where **testing is critical**, such as:
    - Banking systems
    - Medical systems
    - Embedded systems

Model 2: V-Model (Verification & Validation)

Requirements Analysis

System Design

Architectural Design

Module Design

Coding

Acceptance Testing

System Testing

Integration Testing

Unit Testing

TESTING MIRRORS DEVELOPMENT →

Best For:

Safety-critical systems (medical, aerospace)

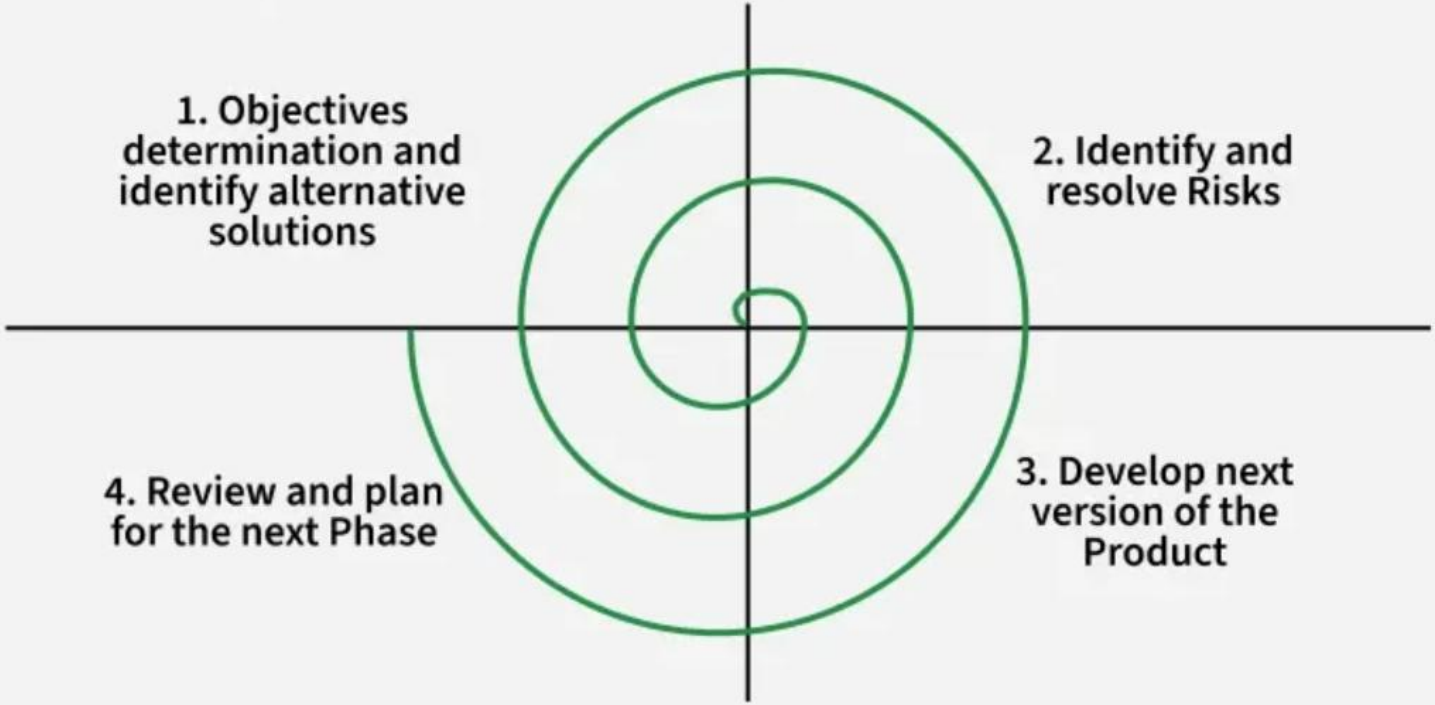
Strength:

Every dev phase has a matching test phase

Weakness:

Inflexible — no iteration after phase lock

Phases of the Spiral Model



Spiral Model

Definition

The **Spiral Model** is a **risk-driven software development model** that combines elements of the **Waterfall Model** and **Iterative development**.
Development happens in **repeated cycles called spirals**, where each cycle focuses on planning, risk analysis, development, and evaluation.

Key Idea

The main focus of the Spiral Model is **risk management**.
Each spiral identifies and reduces risks before moving to the next stage

Advantages

- Good **risk management**.
- Suitable for **large and complex projects**.
- Allows **changes and improvements during development**.
- Continuous **user feedback** improves product quality.

Disadvantages

- **Complex and expensive** to manage.
- Requires **experienced risk analysis experts**.
- Not suitable for **small projects**.
- Development time may be longer.

When to Use Spiral Model

The Spiral Model is suitable when:

- The project is **large and complex**.
- Requirements may **change during development**.
- There is **high risk** involved.
- The system needs **frequent evaluation and testing**.

Model 3 & 4: Incremental + Spiral

Incremental Model

→ Increment 1: Core Features

→ Increment 2: + New Features

→ Increment 3: + More Features

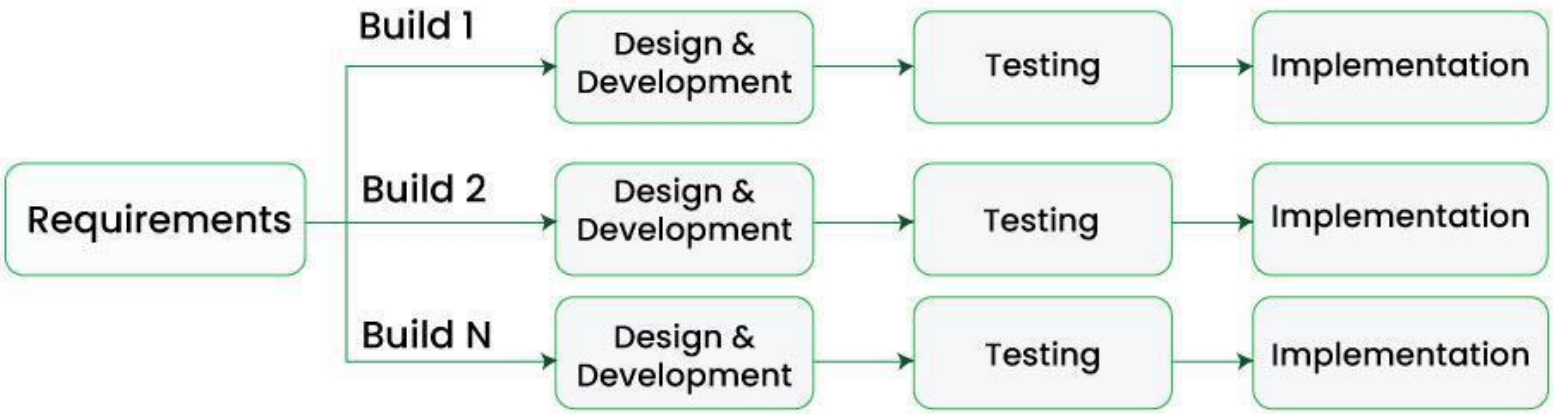
→ Final Release

Flexible · Early delivery · Needs good planning

Spiral Model

1. **Planning**
   Determine objectives, alternatives, constraints
2. **Risk Analysis**
   Identify and resolve risks — prototype if needed
3. **Engineering**
   Develop and test current iteration
4. **Evaluation**
   Customer reviews — plan the next spiral

Risk-driven · Suitable for large complex projects · Costly



Incremental Model

Incremental Model

Definition

The **Incremental Model** is a software development approach where the system is **developed and delivered in small parts called increments**.
Each increment adds **new features and functionality** until the complete system is finished.

Key Idea

Instead of building the entire software at once, the system is **divided into smaller modules**, and each module is developed, tested, and delivered separately.

Advantages

- **Early working software** is delivered to users.
- Easier to **manage smaller modules**.
- **User feedback** can be used in later increments.
- Lower risk compared to building the entire system at once.

Disadvantages

- Requires **good planning and system design**.
- Integration between increments can be complex.
- Not suitable if the **system cannot be divided into modules**.

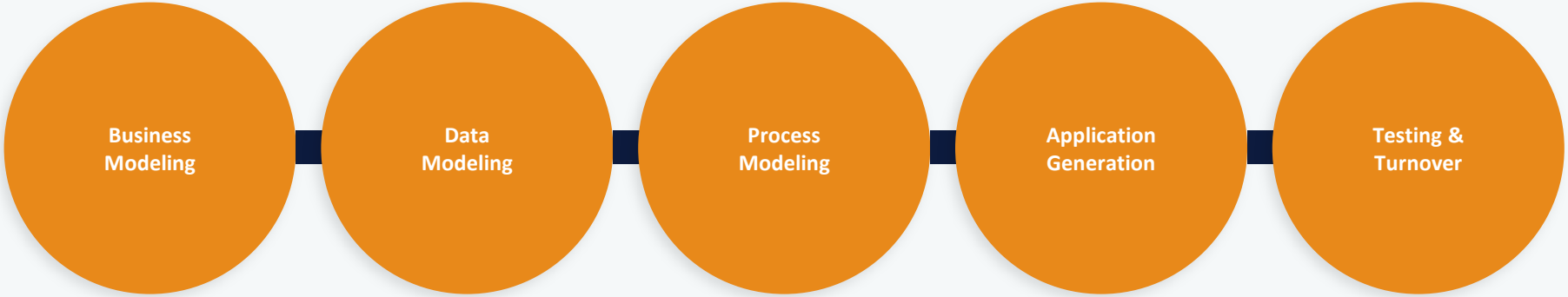
When to Use Incremental Model

The Incremental Model is useful when:

- Requirements are **reasonably well understood**.
- The system can be **divided into modules**.
- Early delivery of **basic functionality is needed**.
- The project is **large and complex**.

Research on RAD and Agile

Model 5: RAD — Rapid Application Development



Why Choose RAD?

Speed

Delivery in 60–90 days using prototypes & reuse

User Involvement

Constant client feedback at every iteration

Reusability

Components and code reused across modules

Component-Based

Parallel development teams on separate modules

Model 6: Agile — The Modern Standard

The 4 Agile Manifesto Values

Individuals & Interactions

over processes and tools

Working Software

over comprehensive documentation

Customer Collaboration

over contract negotiation

Responding to Change

over following a plan

Scrum Sprint Cycle (2–4 Weeks)

1. **Product Backlog**
   Prioritized list of all desired features
2. **Sprint Planning**
   Team selects items for this sprint
3. **Daily Stand-up**
   15-min sync: Done / Doing / Blocked?
4. **Sprint Review**
   Demo the working software to stakeholders
5. **Retrospective**
   What went well? What to improve?

Flexibility: HIGH | Feedback: CONTINUOUS | Speed: FAST

Model Comparison — Choose the Right One

Model	Flexibility	Speed	User Feedback	Best For
Waterfall	Low	Slow	At the End	Fixed requirements (Gov, Defense)
V-Model	Low	Slow	At the End	Safety-critical systems
Incremental	Medium	Medium	Per Increment	Phased feature delivery
Spiral	Medium	Medium	Per Spiral	High-risk, large projects
RAD	High	Very Fast	Continuous	UI-heavy apps, tight deadlines
Agile	Very High	Fast	Every 2–4 weeks	Evolving requirements

Key Takeaways & Session Wrap-Up

1. SDLC gives teams a structured, repeatable approach — no more chaotic development.
2. Waterfall & V-Model work for fixed, well-defined requirements. Don't use them when change is expected.
3. Incremental & Spiral models are great for complex or high-risk projects needing phased delivery.
4. RAD is ideal for fast prototyping — especially for UI-heavy applications with tight deadlines.
5. Agile / Scrum dominates modern development — continuous delivery & fast user feedback loops.
6. DevSecOps is the future — bake security into every SDLC phase, don't bolt it on at the end.

Thank You! 🎉 Based on GeeksforGeeks: Software Development Life Cycle (SDLC)